

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
THE UNIVERSITY OF TEXAS AT ARLINGTON**

**DETAILED DESIGN SPECIFICATION
CSE 4317: SENIOR DESIGN II
SPRING 2025**



**MINDMEAL
MINDMEAL (ADHD FOOD PLANNER)**

**RAKSHYA BASNET
SAUL MORA
UTSHAH NEUPANE
RIKITA KARANJIT
SAINA SHRESTHA
MOHAMMAD HASIBUR RAHMAN**

REVISION HISTORY

Revision	Date	Author(s)	Description
0.1	09.20.2025	RB	Document Creation
0.2	09.22.2025	RB, SM, UN	Introduction Complete
0.3	09.27.2025	RB, SM, UN	System overview complete
1.0	09.28.2025	MHR, RK, SS	First draft
1.1	10.02.2025	MHR, RB, RK, SM, SS, UN	Team Review
1.2	10.09.2025	MHR, RB, RK, SM, SS, UN	Overall Review and Document Completion
1.3	10.10.2025	MHR, RB, RK, SM, SS, UN	Complete Draft

CONTENTS

1	Introduction	5
1.1	Problem Statement	5
1.2	Project Objectives	5
1.3	Purpose of DDS	5
1.4	Design Principles and Constraints	6
1.5	Glossary of Key Terms	6
2	System Overview	6
2.1	System Architecture	6
2.2	Layered Model and Data Flow	7
2.3	Design Rationale	8
2.4	Operational Environment	8
2.5	Key Assumptions and Limitations	9
3	UI Layer Subsystems	10
3.1	Layer Hardware	10
3.2	Layer Operating System	11
3.3	Layer Software Dependencies	11
3.4	Login/Registration	12
3.5	Onboarding	15
3.6	Home Page	18
3.7	Inventory	21
3.8	Recipe Maker (Chat)	24
3.9	Notifications	28
3.10	Camera (Future)	32
4	Application Logic Layer Subsystems	36
4.1	Overview of Application Logic Layer	36
4.2	Layer Hardware	36
4.3	LAYER OPERATING SYSTEM	36
4.4	Layer Software Dependencies	37
4.5	AUTHENTICATION LOGIC	37
4.6	Onboarding Logic	38
4.7	Recipe Engine	39
4.8	Notification Logic	40
4.9	Personal Information Manager	42
5	Data Layer Subsystems	43
5.1	Layer Hardware	43
5.2	Layer Operating System	43
5.3	Layer Software Dependencies	44
5.4	Data Management Subsystem	44
6	Appendix A	47

LIST OF FIGURES

1	System Architecture	7
2	Subsystem Diagram Login Layer	12
3	Subsystem Diagram Onboarding	15
4	Subsystem Diagram Home Page	18
5	Subsystem Diagram Inventory	21
6	Subsystem Diagram Recipe	25
7	Subsystem Diagram Notification	29
8	Subsystem Diagram Camera	32
9	Flow Diagram	36
10	Flow Diagram	44

LIST OF TABLES

2	Key Components and Their Functions in the MindMeal System	44
---	---	----

1 INTRODUCTION

1.1 PROBLEM STATEMENT

Individuals with Attention-Deficit/Hyperactivity Disorder (ADHD) often face challenges with meal planning, consistency, and maintaining dietary habits. Factors like impulsivity, forgetfulness, and difficulty managing time can lead to skipped meals, poor nutrition, or reliance on fast food. MindMeal addresses these challenges through a personalized digital platform that promotes structure and simplicity in daily eating habits. By combining AI-powered recipe suggestions, inventory tracking, and reminder notifications, MindMeal helps users make healthier food decisions while reducing cognitive overload. The project recognizes that conventional meal planners fail to meet the cognitive needs of ADHD users due to complex interfaces, excessive options, and poor feedback loops lead to disengagement. MindMeal's design directly tackles these issues by prioritizing clarity, automation, and personalization.

1.2 PROJECT OBJECTIVES

The main objective of MindMeal is to create a mobile-first ADHD-friendly food planner that simplifies everyday food management through structured yet flexible tools.

Key design and functional objectives:

- Provide personalized recipe recommendations using AI and user inventory.
- Allow simple, fast inventory management (add/update/delete).
- Deliver intelligent notifications for meal reminders and expiring ingredients.
- Support an offline-first experience using local SQLite storage.
- Maintain data synchronization with the backend via secure API calls.
- Design a minimalist, low-distraction interface optimized for ADHD users.
- Ensure scalability for future integration (e.g., barcode scanning, nutrition database).

The app's impact goal is to reduce decision fatigue, increase meal consistency, and support cognitive accessibility for users with ADHD.

1.3 PURPOSE OF DDS

The Detailed Design Specification (DDS) defines the low-level implementation details derived from the Architectural Design Specification (ADS). While the ADS outlined the major components and architecture, this DDS elaborates on how each component is built, including:

- Software modules and classes.
- Data structures, flow, and relationships.
- Algorithms for processing logic.
- Subsystem dependencies and APIs.
- Error-handling, security, and testing methods.

The DDS provides a developer-oriented blueprint, ensuring consistency across the team and serving as a guide for implementation, maintenance, and validation.

1.4 DESIGN PRINCIPLES AND CONSTRAINTS

1.4.1 DESIGN PRINCIPLES

- **Simplicity:** Every screen, function, and decision is reduced to its simplest possible form.
- **Accessibility:** Use of ADHD-friendly design with soft colors, clear hierarchy, consistent feedback, and no flashing elements.
- **Automation:** Wherever possible, the app automates repetitive tasks like recipe generation and reminder scheduling.
- **Modularity:** Layered architecture (UI, Logic, Data) to isolate functionality and ease debugging.
- **Security:** Protection of user data, health info, and storage integrity via encryption and token-based authentication.
- **Offline-first:** SQLite and AsyncStorage ensure usability without constant connectivity.

1.4.2 CONSTRAINTS

- **Performance:** AI-generated recipes depend on API latency; caching and throttling are required.
- **Device Limits:** Must run smoothly on mid-range Android/iOS devices.
- **Privacy:** Personal data (medical, allergies) must comply with data protection policies.
- **Integration Dependencies:** Reliance on third-party APIs (OpenAI, Firebase) introduces service dependencies.

1.5 GLOSSARY OF KEY TERMS

- **ADHD:** Attention Deficit/Hyperactivity Disorder - a neurodevelopmental condition affecting focus and time management.
- **LLM:** Large Language Model - used for intelligent recipe generation and conversation.
- **SQLite:** Lightweight local database for offline storage.
- **AsyncStorage:** React Native API for key-value data persistence.
- **Expo SDK:** Development toolkit for React Native providing access to device features (camera, notifications).
- **JWT (JSON Web Token):** Token-based authentication standard for secure API communication.
- **Material UI:** UI component library for building accessible, consistent interfaces.

2 SYSTEM OVERVIEW

2.1 SYSTEM ARCHITECTURE

The MindMeal application architecture follows a three-tier layered design that separates concerns between user interaction, business logic, and data persistence. As established in the ADS, the system is structured as follows:

- **UI Layer:** Responsible for all user-facing components including Login/Registration, On boarding, Home Page, Inventory, Recipe Maker chat interface, Notifications, and future Camera Module.
- **Application Logic Layer:** Core intelligence of the system such as Authentication Logic, Onboarding Logic, Recipe Engine, Notification Logic, Inventory Management Logic, and Personal Information Manager
- **Data Layer:** Managing all data operations through API Client for external communications, Local Storage for offline-first capabilities, Push Notification Service for alert delivery, and External Integrations for third-party services

Data flows through the system beginning with user interactions in the UI Layer, which trigger appropriate business logic in the Application Logic Layer. For example, when a user requests a recipe suggestion through the Recipe Maker interface, the request passes through the Recipe Engine, which consults both the Inventory Management Logic and the Personal Information Manager to retrieve current food items and dietary restrictions. The Recipe Engine then communicates with external LLM services via the API Client in the Data Layer, processes the response according to allergen filtering rules and dietary goals, and returns personalized recipe cards to the UI for display. Similarly, Inventory Management Logic monitors stored food items in Local Storage and coordinates with Notification Logic to trigger timely alerts about expiring items through the Push Notification Service. This layered architecture ensures modularity, maintainability, and clear separation of concerns while allowing seamless data flow between components to deliver a cohesive user experience focused on reducing cognitive load and supporting consistent eating habits.

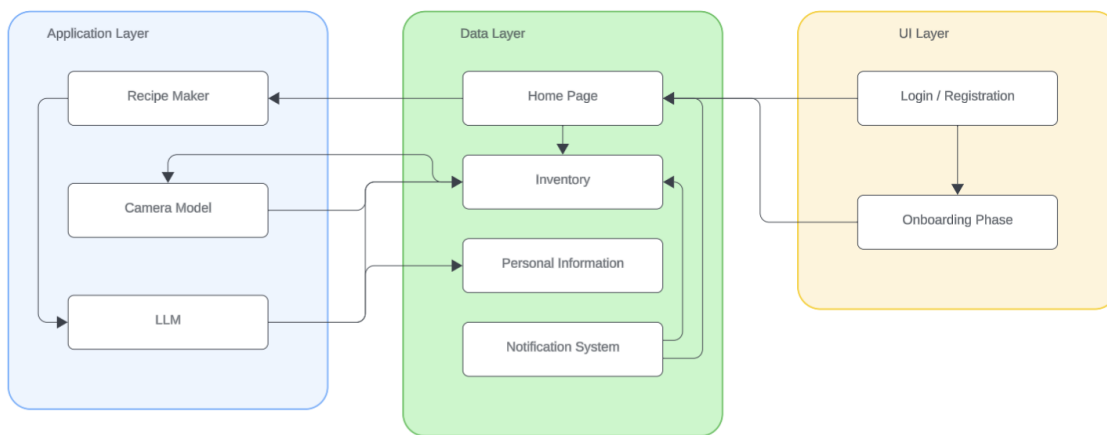


Figure 1: System Architecture

2.2 LAYERED MODEL AND DATA FLOW

Each layer communicates through well-defined interfaces:

UI - Logic Layer

- Data exchange via HTTP requests (Axios).
- Token-based authentication (JWT).
- JSON payloads for all operations.

Logic - Data Layer

- CRUD operations on SQLite through ORM-like access.
- REST/GraphQL APIs to external services (LLM, Nutrition DB).
- Queue-based sync for offline-first support.

Data Layer - UI Layer

- Cached responses for offline use.
- Error responses standardized (HTTP codes + custom messages).
- Notification triggers routed back through Expo Push services.

2.3 DESIGN RATIONALE

The layered design follows the Model-View-Controller (MVC) principle for separation of concerns:

- Model (Data Layer) stores persistent data and handles external APIs.
- View (UI Layer) provides accessible presentation for ADHD users.
- Controller (Logic Layer) governs communication and system rules.

Benefits:

- Reduced coupling between components.
- Independent debugging and testing of each layer.
- Easier scalability (e.g., swapping SQLite for Firebase in future).
- Secure API integration (frontend doesn't directly contact external APIs).

2.4 OPERATIONAL ENVIRONMENT

The MindMeal application operates on Android and iOS mobile platforms, designed for users with varying device specifications and network connectivity. The system requires internet access for real-time synchronization and AI-powered recipe generation but can function in offline mode using locally cached data.

The backend is hosted on the Firebase Cloud Platform, leveraging Firestore for database storage, Authentication for secure access, and Cloud Storage for image uploads. The app frontend is developed using React Native, allowing cross-platform deployment. The operational environment supports Python-based or Node.js backend services for data processing and integration with external APIs.

Supported environments:

- **Mobile Platforms:** Android 8.0+ and iOS 14.0+
- **Network:** Wi-Fi or mobile data (minimum 1 Mbps recommended)
- **Storage:** Minimum 200 MB free space
- **Cloud Infrastructure:** Firebase (Firestore, Auth, Storage, Cloud Messaging)

2.5 KEY ASSUMPTIONS AND LIMITATIONS

Assumptions

- Stable API access for recipe generation.
- Mobile devices have storage access enabled.
- Users provide accurate onboarding data for personalization.

Limitations

- LLM-based suggestions require internet connectivity.
- External API rate limits may throttle high-usage requests.
- Offline mode limited to previously cached data and recipes.
- Future camera integration dependent on Expo Camera API updates.

3 UI LAYER SUBSYSTEMS

The UI Layer represents the presentation layer of the MindMeal application, which includes all user-facing components and interfaces. This layer is implemented as a cross-platform mobile application using React Native, allowing a single code base to target both iOS and Android devices. The UI Layer is responsible for rendering visual components, capturing user input, managing navigation flows, and presenting data received from the Application Logic Layer in an ADHD-friendly format that minimizes cognitive load and decision fatigue.

3.1 LAYER HARDWARE

The UI Layer executes on the mobile device hardware with the following minimum specifications:

iOS Devices:

- iPhone models: iPhone 8 or newer (A-11 Bionic chip or later)
- iPad models: iPad (5th generation) or newer, iPad Air 2 or newer, iPad Pro (all models)
- Minimum RAM: 2 GB
- Storage: Minimum 200 MB of available space for app installation
- Camera: Required for future bar code scanning functionality (8 MP or higher recommended)
- Network: WiFi or cellular data connection (4G/LTE minimum, 5G recommended)
- Display: Minimum 4.7-inch screen (iPhone 8), optimized for displays up to 12.9-inch (iPad Pro)

Android Devices:

- Processor: ARMv8 64-bit or x86-64 architecture
- Minimum RAM: 3 GB
- Storage: Minimum 200 MB of available space for app installation
- Camera: Required for future bar code scanning functionality (8 MP or higher recommended)
- Network: WiFi or cellular data connection (4G/LTE minimum, 5G recommended)
- Display: Minimum 5.0-inch screen, supports resolutions from 720p to 4K
- GPU: OpenGL ES 3.0 support or higher

The UI Layer must support touch input interfaces, including single-tap and swipe gestures. Haptic feedback capabilities are utilized when available to provide tactile confirmation of user actions. The layer does not require any external hardware peripherals for core functionality, though it will integrate with device cameras for the future Camera Module implementation.

3.2 LAYER OPERATING SYSTEM

iOS Platform:

- Minimum supported version: iOS 14.0
- Target version: iOS 17.0 (as of Spring 2025)
- Supports iPhone, iPad, and iPod Touch devices
- Utilize iOS native APIs for push notifications (APNs), local storage (Core Data/User Defaults), and system integrations

Android Platform:

- Minimum supported version: Android 8.0 Oreo (API Level 26)
- Target version: Android 14 (API Level 34)
- Supports smartphones and tablets across multiple manufacturers
- Utilizes Android native APIs for push notifications (Firebase Cloud Messaging), local storage (SQLite/Shared Preferences), and system integrations

The UI Layer is designed to be platform-agnostic at the code level through React Native's abstraction layer, with platform-specific implementations handled through conditional rendering and native modules only where necessary for optimal performance or access to platform-specific features.

3.3 LAYER SOFTWARE DEPENDENCIES

Core Framework:

- **React Native:** Version 0.72.x or higher
 - Cross-platform mobile application framework
 - JavaScript runtime for component logic
 - Native bridge for platform-specific functionality.

UI Component Libraries:

- **React Navigation:** Version 6.x
 - Navigation library for screen transitions
 - Stack, tab, and drawer navigation patterns
 - Deep linking support for notifications

All dependencies are managed through npm (Node Package Manager) and defined in the project's package.json file. Version pinning is used to ensure consistent builds across development environments.

3.4 LOGIN/REGISTRATION

The Login/Registration subsystem is a React Native component-based module that handles user authentication and account creation. It consists of multiple screen components (Login Screen, Registration Screen, Forgot Password Screen) that provide secure access to the MindMeal application. This subsystem interfaces directly with the Authentication Logic in the Application Logic Layer through service abstractions, passing user credentials, and receiving the authentication tokens. The design emphasizes simplicity and ADHD-friendly interactions with clear visual feedback, minimal form fields per screen, and prominent call-to-action buttons to reduce decision fatigue during the authentication process.

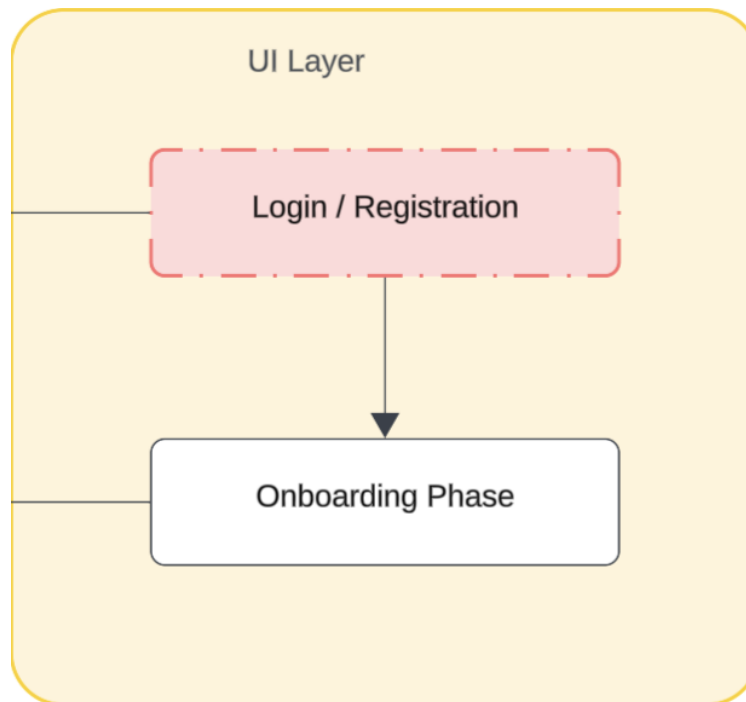


Figure 2: Subsystem Diagram Login Layer

3.4.1 SUBSYSTEM HARDWARE

The Login/Registration subsystem operates on the same mobile device hardware specified in Section 3.1 (Layer Hardware). No additional or specialized hardware is required beyond the capabilities of the base device. The subsystem utilizes:

- **Touchscreen input:** For text entry via on-screen keyboard and button interactions
- **Network interface:** For API communication with the Firebase Authentication service
- **Secure storage:** Device keychain (iOS) or Keystore (Android) for token persistence

Optional biometric hardware (fingerprint sensor, Face ID, facial recognition) may be utilized in future versions for enhanced authentication options but is not required for current implementation.

3.4.2 SUBSYSTEM OPERATING SYSTEM

The Login/Registration subsystem runs on the same operating systems defined in Section 3.2 (Layer Operating System):

- **iOS:** 14.0 or higher
- **Android:** 8.0 (API Level 26) or higher

Platform-specific considerations include the following.

- **iOS:** Utilizes iOS Keychain Services API for secure credential storage
- **Android:** Utilizes Android Keystore system for secure credential storage
- Both platforms support background token refresh capabilities through OS-level background task scheduling.

3.4.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Authentication Services:

- **Clerk:**
 - Email/password authentication
 - Token management and refresh
 - Password reset functionality

Security Libraries:

- **react-native-keychain:** Version 8.x
 - Secure storage of authentication tokens
 - Platform-agnostic API for credential storage
 - Biometric authentication support (future)

Form Components:

- **React Hook Form:** Version 7.x (shared with layer dependencies)
 - Form state management
 - Input validation (email format, password strength)
 - Error message display

UI Components:

- **React Native Paper:** Text inputs, buttons, and cards (shared with layer dependencies)
- **react-native-vector-icons:** Icons for password visibility toggle and social login buttons

Navigation:

- **React Navigation:** Screen transitions between Login, Register, and Forgot Password screens (shared with layer dependencies)

The subsystem maintains minimal dependencies to ensure fast load times and reduce potential security vulnerabilities in the authentication flow.

3.4.4 SUBSYSTEM PROGRAMMING LANGUAGES

Primary Language:

- **JavaScript/TypeScript:** Version ECMAScript 2022 (ES13) with TypeScript 5.x
 - Component logic and event handling
 - Form validation and state management
 - API communication with Firebase Authentication
 - TypeScript provides static type checking for improved code quality and developer experience

Markup and Styling:

- **JSX/TSX:** React's XML-like syntax for component templates
- **JavaScript (Inline Styles):** React Native styling through JavaScript objects
- **CSS-in-JS:** Style definitions using React Native's StyleSheet API

Configuration:

- **JSON:** Configuration files and app settings

3.4.5 SUBSYSTEM DATA STRUCTURES

- Login Credentials & Register Credentials interfaces - Input data structures
- AuthState & User interfaces - Application state management
- AuthError interface - Error handling structure
- Form validation schema - Validation rules
- API Request/Response structures - Authentication payloads

3.4.6 SUBSYSTEM DATA PROCESSING

Security Processing:

- All passwords are transmitted over HTTPS only
- Passwords are never stored locally (only authentication tokens)
- Rate limiting enforced by CLERK (prevents brute force attacks)
- Failed login attempts are tracked and trigger temporary account lockout after 5 consecutive failures
- Email verification required before full account access
- Biometric authentication (future) will use device secure enclave

Input Validation:

- Validate email format using regex pattern matching
- Check password length (minimum 8 characters, maximum 128 characters)

- Sanitize input to prevent XSS attacks
- Display field-level validation errors in real-time

Error Handling Strategy:

- Network errors trigger retry logic with exponential backoff (1s, 2s, 4s, 8s)
- Timeout set to 30 seconds for authentication requests
- Graceful degradation: if CLERK is unavailable, display maintenance message
- All errors logged to analytics service for monitoring

Performance Optimization:

- Token stored in memory after initial login to avoid repeated Keychain/Keystore access
- Token refresh occurs in background 5 minutes before expiration
- Login form inputs debounced to reduce validation calls

3.5 ONBOARDING

The Onboarding subsystem is a multi-step wizard component that collects essential personal and health information from first-time users. Implemented as a series of React Native screen components, this subsystem guides users through a progressive disclosure of information gathering, including age, medical conditions, allergies, dietary goals, and ADHD-specific preferences. Each step presents a focused set of inputs to avoid overwhelming users, with progress indicators providing clear feedback on the completion status. The collected data are temporarily stored in the component state and persist to local storage upon completion, then synchronized with backend services through the Personal Information Manager in the Application Logic Layer.

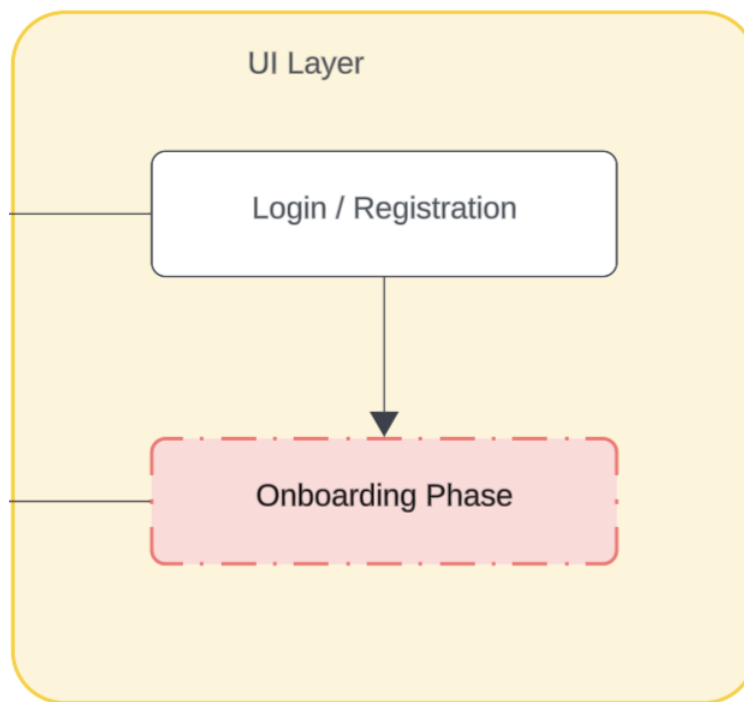


Figure 3: Subsystem Diagram Onboarding

3.5.1 SUBSYSTEM HARDWARE

The Onboarding subsystem operates on standard mobile device hardware as specified in Section 3.1 (Layer Hardware). No specialized hardware is required. The subsystem utilizes:

- **Touchscreen input:** For text entry, checkbox selection, and button interactions across multi-step forms
- **Local storage:** Device storage for caching onboarding progress in case of interruption
- **Network interface:** For synchronizing completed profile data with backend services

The subsystem is designed to function entirely offline during data collection, with network connectivity required only for final data synchronization.

3.5.2 SUBSYSTEM OPERATING SYSTEM

The Onboarding subsystem runs on the same operating systems defined in Section 3.2 (Layer Operating System):

- **iOS:** 14.0 or higher
- **Android:** 8.0 (API Level 26) or higher

No platform-specific operating system features are required beyond standard React Native support. The subsystem maintains consistent behavior on both platforms.

3.5.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Form Management:

- **React Hook Form:** Version 7.x (shared with layer dependencies)
 - Multi-step form state management
 - Wizard navigation with data persistence between steps
 - Field-level validation

UI Components:

- **React Native Paper:** Check boxes, radio buttons, text inputs, progress indicators (shared with layer dependencies)
- **React-native-picker:** Version 2.x for drop down selections (blood group, dietary preferences)
- **React-native-step-indicator:** Version 1.x for visual progress tracking across onboarding steps

Storage:

- **AsyncStorage:** Version 1.19.x (shared with layer dependencies)
 - Persisting partial onboarding data for resume capability
 - Caching completed profile until backend sync

Data Structure:

- **TypeScript interfaces:** Define strict types for personal information schema

- Ensures type safety for health-related data
- Compile-time validation of data structures

The subsystem emphasizes simplicity in dependencies to maintain fast rendering and minimize cognitive load during the critical first-user experience.

3.5.4 SUBSYSTEM PROGRAMMING LANGUAGES

Primary Language:

- **JavaScript/TypeScript:** Version ECMAScript 2022 (ES13) with TypeScript 5.x
 - Multi-step wizard logic and state management
 - Form validation across onboarding steps
 - Data serialization for profile creation
 - TypeScript interfaces ensure type safety for health data

Markup and Styling:

- **JSX/TSX:** React Native component templates for onboarding screens
- **JavaScript (StyleSheet API):** React Native styling for step indicators, form inputs, and buttons

Configuration:

- **JSON:** Configuration of the Onboarding question and validation rules

All Onboarding logic is implemented in TypeScript to prevent data type errors when collecting sensitive health information.

3.5.5 SUBSYSTEM DATA STRUCTURES

The Onboarding subsystem manages user-provided data through structured TypeScript interfaces that ensure type safety and consistency across modules. Each interface models a specific category of information collected during the onboarding process. The structures are validated before being persisted to local storage or transmitted to backend services.

3.5.6 SUBSYSTEM DATA PROCESSING

The Onboarding subsystem implements a multi-step data collection algorithm using progressive disclosure to reduce cognitive load for ADHD users. Data processing follows these main stages:

Initialization:

- The wizard initializes the form state with empty or cached values from AsyncStorage.
- The validation rules (defined in the JSON configuration) are dynamically loaded .

Step Validation & Navigation:

- Each Onboarding step uses React Hook Form with schema-based validation (Yup).
- In “Next,” the input in the current step is validated; the valid data merge into the cumulative form state.
- Invalid inputs trigger visual feedback and haptic vibration (optional).

Local Caching Algorithm:

- After each step, the partial data are serialized and stored using `AsyncStorage.setItem()`.
- If the app closes unexpectedly, the process resumes at the last saved step using a progress key.

Profile Assembly:

- Upon completion, all step data are merged into a single Personal Profile object.
- This object is validated against TypeScript types and a secondary runtime schema prior to synchronization.

Synchronization Strategy:

- When online, the final JSON payload is transmitted via the Personal Information Manager to backend services using HTTPS POST.
- In case of a failure, a retry policy with exponential back-off is applied (1s, 2s, 4s, up to 30s).

Security & Error Handling:

- Sensitive fields (e.g., medical conditions) are encrypted at rest using AES-256 prior to local storage.
- Error logs are anonymized and stored for analysis (if enabled).

3.6 HOME PAGE

The Home Page subsystem serves as the central dashboard and navigation hub of the MindMeal application. Implemented as a React Native functional component, it provides an at-a-glance overview of recent activity including recent recipe chats, upcoming meal suggestions, and active notifications. The interface follows ADHD-friendly design principles with a clean layout, prominent navigation buttons limited to 3-4 primary actions, and a search bar for quick access to specific features. The Home Page coordinates with multiple Application Logic Layer components to display real-time data, including recent chat history from the Recipe Engine, notification counts from the Notification Logic, and inventory status from the Inventory Management Logic.

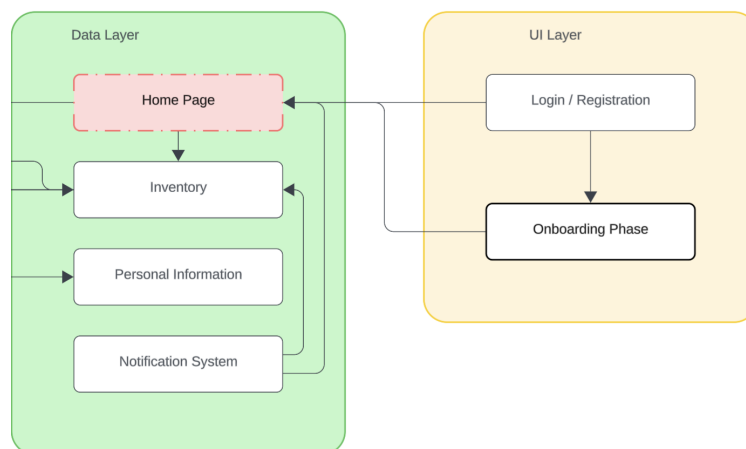


Figure 4: Subsystem Diagram Home Page

3.6.1 SUBSYSTEM HARDWARE

The Home Page subsystem operates on standard mobile device hardware as specified in Section 3.1 (Layer Hardware). No specialized hardware is required. The subsystem utilizes:

- **Touchscreen input:** For navigation button taps, search bar interaction, and scrolling through recent chats
- **Display:** Render responsive layouts optimized for various screen sizes
- **Network interface:** For real-time data synchronization and loading recent content

The subsystem is optimized for smooth scrolling performance in the minimum specified hardware configurations.

3.6.2 SUBSYSTEM OPERATING SYSTEM

The Home Page subsystem runs on the same operating systems defined in Section 3.2 (Layer Operating System):

- **iOS:** 14.0 or higher
- **Android:** 8.0 (API Level 26) or higher

The subsystem utilizes standard React Native rendering without platform-specific requirements, ensuring consistent appearance and behavior across both operating systems.

3.6.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Navigation:

- **React Navigation:** Version 6.x (shared with layer dependencies)
 - Tab navigation for Home, Inventory, Notifications sections
 - Stack navigation for deep linking into Recipe Maker and other features
 - Header customization for search bar integration

UI Components:

- **React Native Paper:** Cards, buttons, search bar, FAB (Floating Action Button) (shared with layer dependencies)
- **React-native-vector-icons:** Navigation icons and action buttons (shared with layer dependencies)

List Rendering:

- **FlatList:** React Native built-in component
 - Optimized rendering of recent recipe chat list
 - Lazy loading and virtualization for performance
 - Pull-to-refresh functionality

State Management:

- **React Context API:** (shared with layer dependencies)
 - Global state for user profile data
 - Notification badge counts
 - Recent activity synchronization

Data Fetching:

- **React Query (optional):** Version 5.x
 - Caching and synchronization of recent chats
 - Automatic background refresh
 - Optimistic updates for better UX

Date/Time Utilities:

- Formatting timestamps for recent activity
- Relative time display

The Home Page maintains minimal direct dependencies, primarily leveraging shared layer dependencies to ensure fast initial load times and smooth interactions.

3.6.4 SUBSYSTEM PROGRAMMING LANGUAGES

- JavaScript/TypeScript (ES2022 with TypeScript 5.x)
- JSX/TSX for React Native components
- JavaScript StyleSheet API for styling
- JSON for configuration

3.6.5 SUBSYSTEM DATA STRUCTURES

- DashboardState interface - main dashboard state
- RecipeChat interface - recent chat data
- NotificationCount interface - badge counts
- InventoryStatus interface - inventory overview
- MealSuggestion interface - meal recommendations
- NavigationItem and QuickAction interfaces - navigation data
- SearchState and SearchResult interfaces - search functionality

3.6.6 SUBSYSTEM DATA PROCESSING

- Dashboard Initialization Algorithm
- Data Aggregation and Refresh Algorithm
- Greeting Generation Algorithm (time-based personalized greetings)
- Search Processing Algorithm (with debouncing and relevance ranking)
- Quick Action Handling
- Performance optimizations and ADHD-friendly features

3.7 INVENTORY

The Inventory subsystem is a CRUD (Create, Read, Update, Delete) interface component that allows users to manage their food items. Implemented as a React Native screen component with modal overlays, it displays food items in a table/list format with quick actions for editing and removing items. A floating action button (FAB) provides immediate access to add new inventory items. The subsystem interfaces with the Inventory Management Logic in the Application Logic Layer for data operations and the Notification Logic for expiring item alerts. The design emphasizes visual clarity with thumbnails of items, highlighting expiration date, and minimal-click interactions to accommodate ADHD-related executive function challenges.

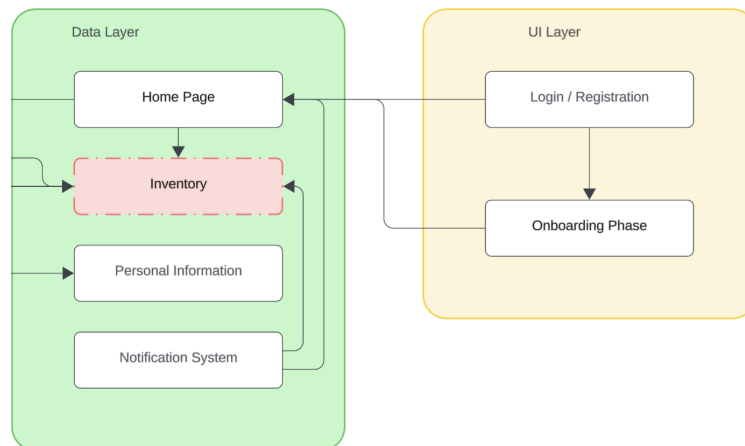


Figure 5: Subsystem Diagram Inventory

3.7.1 SUBSYSTEM HARDWARE

The Inventory subsystem operates on standard mobile device hardware as specified in Section 3.1 (Layer Hardware). The hardware used includes:

- **Touchscreen input:** To add, edit, and delete inventory items through touch interactions
- **Local storage:** For offline-first inventory data persistence
- **Camera (future enhancement):** For capturing food item photos to aid visual memory
- **Network interface:** For synchronizing inventory data with cloud services

The subsystem is designed to function fully offline with automatic synchronization when network connectivity is restored

3.7.2 SUBSYSTEM OPERATING SYSTEM

The Inventory subsystem runs on the same operating systems defined in Section 3.2 (Layer Operating System)

- **iOS:** 14.0 or higher
- **Android:** 8.0 (API Level 26) or higher

Platform-specific considerations include:

- **iOS:** Utilizes iOS native date picker for expiration date selection
- **Android:** Utilizes Android Material date picker for expiration date selection

Both platforms support local SQLite database access for inventory storage.

3.7.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Database:

- **React-native-sqlite-storage:** Version 6.x
 - Local SQLite database for inventory persistence
 - CRUD operations for food items
 - Query support for filtering and sorting
 - Migration support for schema updates

UI Components:

- **React Native Paper:** FAB, Data Table, Modal, Text Input, Button (shared with layer dependencies)
- **react-native-vector-icons:** Icons for add, edit, delete actions (shared with layer dependencies)
- **react-native-modal:** Version 13.x for add/edit item dialog

Form Handling:

- **React Hook Form:** Version 7.x (shared with layer dependencies)
 - Add/edit item form management
 - Validation for required fields (item name, quantity)

Date Handling:

- **date-fns:** Version 3.x (shared with layer dependencies)
 - Date parsing and formatting for expiration dates
 - Date comparison for expiration item detection
- **React-native-date-picker:** Version 4.x
 - Native date picker for expiration date selection
 - Platform-specific implementations

List Management:

- **FlatList:** React Native built-in component
 - Optimized rendering of inventory items
 - Swipe-to-delete gesture support
 - Search and filter capabilities

Image Handling (future):

- **React-native-image-picker:** Version 5.x for item photos

- **React-native-fast-image:** Version 8.x for optimized image loading and caching

State Synchronization:

- **AsyncStorage:** Version 1.19.x (shared with layer dependencies)
 - Caching inventory changes for offline operation
 - Sync queue for pending updates

The subsystem emphasizes on offline-first architecture with robust local storage to ensure that inventory remains accessible regardless of network conditions.

3.7.4 SUBSYSTEM PROGRAMMING LANGUAGES

Primary Languages:

- **JavaScript/TypeScript:** Version ECMAScript 2022 (ES13) with TypeScript 5.x
 - Defines data interfaces for inventory items
 - Implements CRUD logic for local and synchronized operations
 - Handles event-driven updates via state management hooks

Used for all logic and UI components within the subsystem. TypeScript ensures strong typing of inventory objects, form validation, and database queries, reducing runtime errors and improving maintainability.

Markup and Styling:

- **JSX/TSX:** Used to define React Native component templates for the inventory list, modal dialogs, and buttons.
- **React Native StyleSheet API:** Used for defining consistent styling across list rows, form inputs, and FAB elements.

Configuration:

- **JSON:** Used to define the default category list, validation rules, and initial seed data for the SQLite schema

3.7.5 SUBSYSTEM DATA STRUCTURES

The Inventory subsystem relies on structured TypeScript interfaces and SQLite table schemas to maintain the consistency and integrity of stored food item data. These structures define how items are represented both in-memory and in persistent storage.

3.7.6 SUBSYSTEM DATA PROCESSING

The Inventory subsystem implements data processing strategies focused on offline-first CRUD operations, expiration detection, and notification triggering.

- **CRUD Processing Algorithm:** Inventory data processing follows a local-first, eventual-consistency model.
 - **Create:**
 - * User inputs item details via modal form.

- * Data validated using React Hook Form rules (required fields: *name*, *quantity*, *expirationDate* (optional)).
- * Item inserted into SQLite database with a generated UUID.
- * Item added to pending sync queue if offline.
- **Read:**
 - * On screen load, items are fetched using an optimized


```
SELECT * FROM inventory ORDER BY expiration_date;
```
 - * Item expired or near-expired are visually highlighted.
- **Update:**
 - * The user edits the fields via the modal form.
 - * Updates are applied locally using SQL *UPDATE* statements and are cached in the sync queue.
- **Delete:**
 - * Swipe-to-delete gesture triggers a soft-delete flag in SQLite:


```
DELETE FROM inventory WHERE id=?
```
 - * The corresponding deletion added to the sync queue.
- **Synchronization:**
 - * When online, a background sync service transmits queued updates to the backend API via HTTPS POST.
 - * Successful responses mark records as *synced=true*.
- **Expiration Detection Algorithm:** To minimize cognitive overload, expiration alerts are automatically generated without user input.
 - **Algorithm Logic**
 - * Uses date-fns for date comparison.
 - * Items within a 48-hour window of expiration trigger the Notification Logic.
 - * Each alert updates the record (*alertSent = true*) to prevent duplicates
- **Synchronization Algorithm:**
 - The subsystem follows an exponential backoff retry for failed sync operations

3.8 RECIPE MAKER (CHAT)

The Recipe Maker subsystem provides a conversational chat interface where users can request personalized meal suggestions. Implemented as a React Native screen component with a scrollable message list and text input field, it mimics familiar messaging applications while integrating AI-powered recipe generation. Users type natural language queries (e.g., "What can I make for dinner?"), and the subsystem communicates with the Recipe Engine in the Application Logic Layer, which processes the request using current inventory, dietary restrictions, and personal health information. The responses are displayed as formatted recipe cards within the chat thread, showing ingredients, preparation time, cooking instructions, and allergen warnings. The interface maintains a conversation history for easy reference to previously suggested meals.

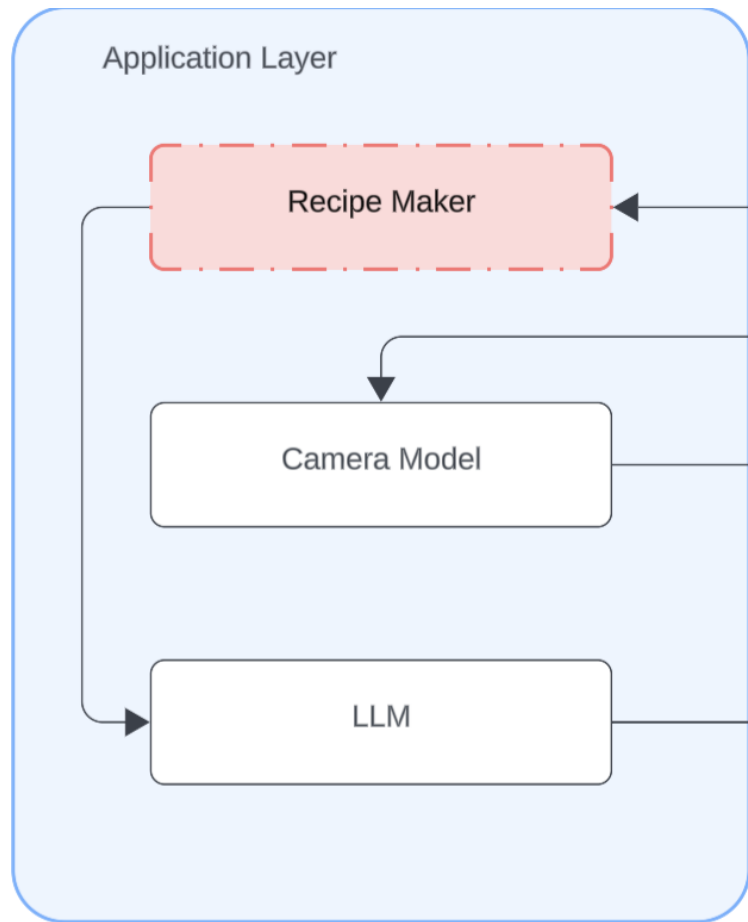


Figure 6: Subsystem Diagram Recipe

3.8.1 SUBSYSTEM HARDWARE

The Recipe Maker subsystem operates on standard mobile device hardware as specified in Section 3.1 (Layer Hardware). The hardware used includes:

- **Touchscreen input:** For text entry via on-screen keyboard and scrolling through chat history
- **Display:** Renders chat bubbles, recipe cards, and inline images with responsive layout
- **Network interface:** Required for real-time communication with LLM-based Recipe Engine API
- **Local storage:** For caching chat history and recipe suggestions for offline viewing

The subsystem requires active network connectivity to generate new recipe suggestions, but allows offline access to previously generated recipes stored locally.

3.8.2 SUBSYSTEM OPERATING SYSTEM

The Recipe Maker subsystem runs on the same operating systems defined in Section 3.2 (Layer Operating System):

- **iOS:** 14.0 or higher

- **Android:** 8.0 (API Level 26) or higher

Both platforms support the necessary APIs for chat-style interfaces, keyboard management, and scrollable content. No platform-specific features are required.

3.8.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Chat Interface:

- **react-native-gifted-chat:** Version 2.x
 - Pre-built chat UI components
 - Message bubbles, input toolbar, and send button
 - Typing indicators and timestamp formatting
 - Image and custom message rendering support

State Management:

- **React Context API or Redux Toolkit:** (shared with layer dependencies)
 - Managing chat message state
 - User input state
 - Loading states during recipe generation

HTTP Communication:

- **Axios:** Version 1.x
 - HTTP client for API requests to Recipe Engine
 - Request/response interceptors for authentication
 - Timeout and retry logic for network resilience

Message Storage:

- **AsyncStorage:** Version 1.19.x (shared with layer dependencies)
 - Persisting chat history locally
 - Caching generated recipes for offline access
 - Sync with cloud storage for cross-device access

Markdown Rendering:

- **react-native-markdown-display:** Version 7.x
 - Rendering formatted recipe instructions with headings, lists, and bold text
 - Custom styling for ADHD-friendly readability

UI Components:

- **React Native Paper:** Recipe card components, buttons for actions (shared with layer dependencies)

- **React-native-vector-icons:** Icons for send button, ingredient checkmarks (shared with layer dependencies)

Date/Time:

- **date-fns:** Version 3.x (shared with layer dependencies)
 - Formatting message timestamps
 - Sorting messages chronologically

Input Management:

- **React-native-keyboard-aware-scroll-view:** Version 0.9.x
 - Automatic scrolling when keyboard appears
 - Prevents input field from being hidden by keyboard

The subsystem leverages established chat libraries to provide a familiar user experience while integrating custom recipe card rendering for optimal presentation of meal suggestions.

3.8.4 SUBSYSTEM PROGRAMMING LANGUAGES

Primary Language:

- **TypeScript (v5.x):** Used throughout the subsystem for chat interface logic, API communication, and message state management. TypeScript's strong typing ensures message consistency, safe API calls, and predictable data handling between chat messages, recipe cards, and backend responses.
 - Type-safe message and recipe models.
 - Integration with the Recipe Engine API using asynchronous Axios requests.
 - Controlled state updates through the Context API

Markup and Styling:

- **JSX/TSX:** Used to define component structures such as chat bubbles, input toolbars, and recipe cards within the Gifted Chat interface.
- **React Native StyleSheet API:** Handles component-level styling for ADHD-friendly layouts (increased spacing, large fonts, minimal color distractions).
- **Markdown:** Recipe instructions and cooking steps are rendered using Markdown syntax for a formatted presentation (bold text, bullet points, and step numbering).

Configuration Files:

- **JSON:** Defines API routes, retry intervals, and default user prompts (e.g., "Suggest breakfast ideas").

This combination of TypeScript, JSX, and Markdown ensures maintainability, accessibility, and readability within the chat-based recipe suggestion experience.

3.8.5 SUBSYSTEM DATA STRUCTURES

The Recipe Maker subsystem maintains structured message and recipe data through defined TypeScript interfaces. These data structures enable seamless interaction between the chat interface, the Recipe Engine API, and local storage.

3.8.6 SUBSYSTEM DATA PROCESSING

The Recipe Maker subsystem processes natural language queries through a combination of message handling, recipe retrieval, and rendering pipelines. Its algorithms emphasize responsiveness, type safety, and ADHD-friendly flow continuity.

Message Handling Algorithm

- **User Input Event**
 - The user submits text through the Gifted Chat input bar.
 - Input is validated to prevent empty or duplicate messages.
 - The message is appended to the chat state and displayed immediately as a “user” message.
- **API Request to Recipe Engine**
 - Axios sends a *POST* request to the Recipe Engine API with timeout and retry logic
- **Error Recovery**
 - If the API call fails, the subsystem retries with exponential backoff (1s, 2s, 4s, 8s).
 - If all attempts fail, a fallback message is displayed: *“I couldn’t fetch new recipes right now. Try again once you’re online.”*

Recipe Rendering Pipeline

The rendering process transforms raw JSON recipe data into a visually structured card using Markdown formatting

- Parse JSON response to populate Recipe Card interface.
- Format ingredients as bulleted lists.
- Apply Markdown styling (headings for steps, bold for ingredient names).
- Render recipe card using *React-native-markdown-display*.

This ensures that recipes are visually appealing and easily digestible for ADHD users, reducing information overload.

Conversation Persistence Algorithm

- Every message update triggers an auto-save to AsyncStorage.
- On app relaunch, cached messages are reloaded and displayed, maintaining chat continuity.
- Older chats may be archived or synced to cloud storage for space optimization.

3.9 NOTIFICATIONS

The Notifications subsystem displays alerts and reminders to users in a centralized, organized interface. Implemented as a React Native screen component with tabbed navigation, it groups notifications into categories (inventory alerts, recipe prompts, custom reminders) to reduce cognitive overwhelm. Each notification displays relevant information (message, timestamp, category) with swipe-to-dismiss functionality and action buttons where appropriate (e.g., "View Recipe" for recipe prompts, "Check Inventory" for expiring item alerts). The subsystem receives notification data from the Notification Logic in the Application Logic Layer and coordinates with the Push Notification Service for system-level alert delivery. Unread notifications are badged on the Home Page navigation icon to draw appropriate attention.

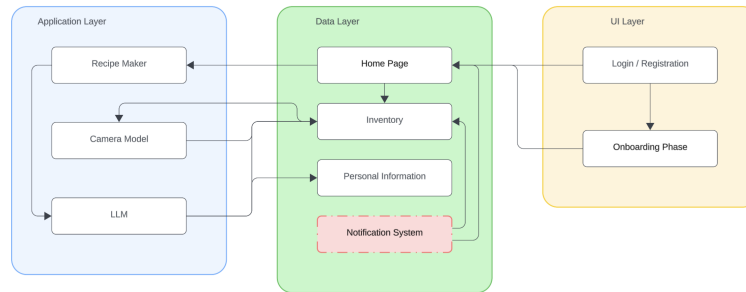


Figure 7: Subsystem Diagram Notification

3.9.1 SUBSYSTEM HARDWARE

The Notifications subsystem operates on standard mobile device hardware as specified in Section 3.1 (Layer Hardware). The hardware used includes:

- **Touchscreen input:** For tab selection, notification dismissal gestures, and action button taps
- **Display:** Renders notification cards with color-coded categories and urgency indicators
- **Haptic feedback (optional):** Provides tactile confirmation when dismissing notifications
- **Network interface:** For syncing notification read/unread status across devices

The subsystem functions fully offline, with notification status synchronization occurring when network connectivity is available.

3.9.2 SUBSYSTEM OPERATING SYSTEM

The Notifications subsystem runs on the same operating systems defined in Section 3.2 (Layer Operating System):

- **iOS:** 14.0 or higher
 - Integrates with iOS Notification Center for system-level alerts
 - Supports notification badges on app icon
- **Android:** 8.0 (API Level 26) or higher
 - Integrates with Android Notification Drawer for system-level alerts
 - Supports notification badges and channels

Platform-specific notification styling follows native conventions (iOS notification banners vs. Android notification cards).

3.9.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Push Notifications:

- **react-native-firebase:** Version 18.x
 - Firebase Cloud Messaging (FCM) integration
 - Push notification reception and handling

- Background and foreground notification management
- Notification token management

Local Notifications:

- **notifee/react-native:** Version 7.x
 - Local notification scheduling
 - Custom notification styling and actions
 - Notification channels (Android) and categories (iOS)
 - Badge management

UI Components:

- **React Native Paper:** Cards, badges, tabs (shared with layer dependencies)
- **React-native-vector-icons:** Category icons (inventory, recipe, reminder) (shared with layer dependencies)
- **React-native-tab-view:** Version 3.x
 - Tabbed interface for notification categories
 - Swipeable tabs for easy navigation

List Rendering:

- **FlatList:** React Native built-in component
 - Optimized rendering of notification list
 - Pull-to-refresh for fetching latest notifications
 - Empty state handling

Storage:

- **AsyncStorage:** Version 1.19.x (shared with layer dependencies)
 - Persisting notification history
 - Tracking read/unread status
 - Storing dismissed notification IDs

Navigation:

- **React Navigation:** Version 6.x (shared with layer dependencies)
 - Deep linking from notifications to specific screens
 - Navigation from notification actions to Inventory or Recipe Maker

3.9.4 SUBSYSTEM PROGRAMMING LANGUAGES

The Notifications subsystem is developed using the same cross-platform language stack as the broader application, optimized for React Native mobile development:

- **JavaScript (ES2021):** Primary language for component logic, styling, and data handling within React Native.
- **TypeScript (4.x):** Used for type safety and maintainability in larger modules, especially for notification object structures, interface definitions, and navigation props.

React Native bridges abstract most native code interaction, allowing the majority of logic to remain within the JavaScript/TypeScript layer.

3.9.5 SUBSYSTEM DATA STRUCTURES

Notifications in the system are structured into a consistent and serializable object format to ensure compatibility across platform APIs and local storage. The core data structure is the Notification Item, defined in TypeScript as follows:

- **Notification Category:** Enum for tab display logic (*inventory, recipe, reminder*)
- **Notification Store:** In-memory state store (context or Redux) that tracks all current notifications, read/unread status, and dismissed IDs.
- **Notification Token:** A string token used for push notification registration and device identification via Firebase Cloud Messaging (FCM).

Notifications are stored as serialized JSON arrays in AsyncStorage and rehydrated on app launch.

3.9.6 SUBSYSTEM DATA PROCESSING

Notification data undergoes multiple stages of processing, from reception to UI rendering and user interaction. Key data processing strategies include:

- **Push Notification Handling:**
 - On message receipt (via Firebase), a notification is parsed, validated, and optionally displayed using notifee.
 - If the app is in the foreground, the notification is routed to the in-app display queue and handled contextually (e.g., a toast or modal).
- **Notification Categorization:**
 - Notifications are filtered into categories (*inventory, recipe, reminder*) on load using simple predicate functions.
 - Category filters also support tabbed rendering and badge counts.
- **Swipe-to-Dismiss Gesture Recognition:**
 - Implemented using gesture handler libraries ('React-native-gesture-handler') to remove notifications from view and flag their 'dismissed' state in local storage.
- **Read/Unread State Management:**

- Notifications are marked as read when tapped or opened.
- Read state updates are stored locally and synced to the backend when online.
- Badge counts on the Home tab are recalculated based on unread items.

Notification processing is designed to be resilient to app state (background, kill, foreground) and provides a seamless user experience whether the alert is system-level or in-app.

3.10 CAMERA (FUTURE)

The Camera Module subsystem is planned for future implementation to enable barcode and food label scanning functionality. When completed, it will be implemented as a React Native screen component that activates the device camera and provides real-time viewfinder display with overlay graphics for scan targeting. Upon successful barcode recognition, the module will query external nutrition databases to retrieve product information and automatically populate inventory fields. Currently, a placeholder screen is displayed indicating "Coming Soon" functionality. The design will emphasize quick capture and minimal user interaction, allowing users to rapidly add multiple items to inventory through successive scans without returning to previous screens.

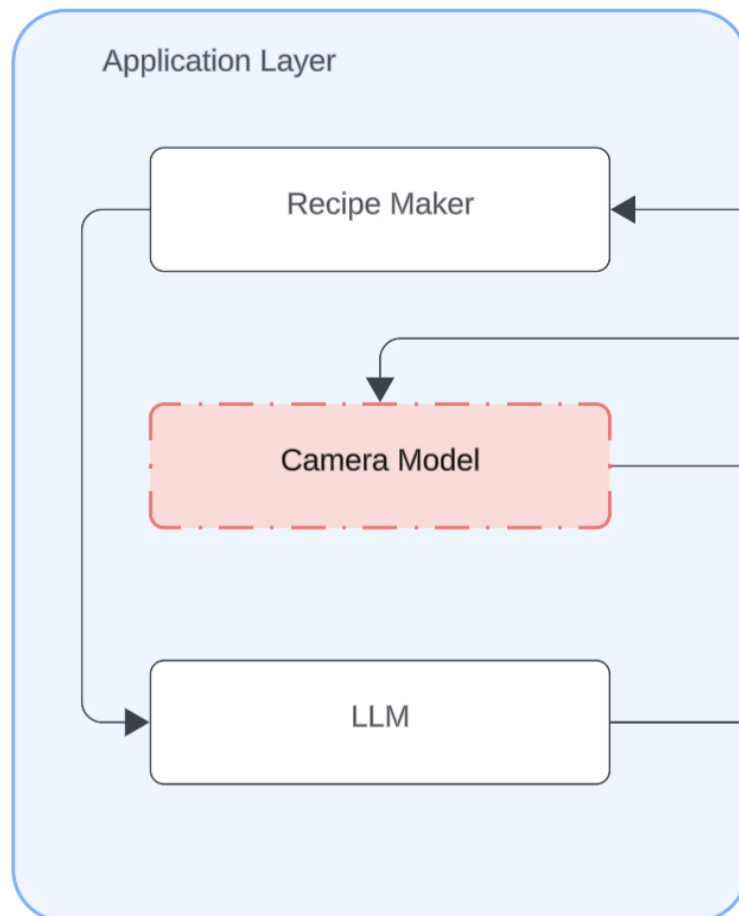


Figure 8: Subsystem Diagram Camera

3.10.1 SUBSYSTEM HARDWARE

The Camera Module subsystem will require the following hardware capabilities:

- **Rear camera:** Minimum 8MP resolution for reliable barcode scanning
- **Autofocus:** Required for clear barcode image capture
- **LED flash:** Optional but recommended for low-light scanning
- **Touchscreen input:** For manual focus adjustment and capture confirmation
- **Network interface:** For querying external bar code databases (UPC Database API, Open Food Facts API)
- **Local storage:** For caching scanned product information

The subsystem will be designed to gracefully handle devices without camera access by providing manual entry alternatives.

3.10.2 SUBSYSTEM OPERATING SYSTEM

The Camera Module subsystem will run on the same operating systems defined in Section 3.2 (Layer Operating System):

- **iOS:** 14.0 or higher
 - Requires camera permissions via Info.plist configuration
 - Utilizes AVFoundation framework for camera access
- **Android:** 8.0 (API Level 26) or higher
 - Requires camera permissions via AndroidManifest.xml
 - Utilizes CameraX API for camera access

Both platforms require runtime permission requests before camera access is granted.

3.10.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Camera Access:

- **React-native-vision-camera:** Version 3.x
 - High-performance camera library with barcode scanning support
 - Real-time frame processing for barcode detection
 - Camera device selection and configuration
 - Flash and focus control

Bar-code Scanning:

- **Vision-camera-code-scanner:** Version 0.2.x
 - Vision Camera plugin for barcode detection
 - Supports multiple barcode formats (UPC-A, EAN-13, QR codes)
 - Real-time scanning without frame capture

Image Processing:

- **React-native-image-resizer:** Version 3.x

- Image optimization before uploading for label recognition
- Reduces bandwidth usage for API calls
- **React-native-image-crop-picker:** Version 0.40.x
 - Manual image capture and cropping for difficult bar codes
 - Gallery access for scanning product photos

API Communication:

- **Axios:** Version 1.x (shared with Recipe Maker dependencies)
 - HTTP requests to barcode lookup services
 - Nutrition database API queries

Permissions:

- **react-native-permissions:** Version 4.x
 - Cross-platform permission request handling
 - Camera permission status checking

UI Components:

- **React Native Paper:** Buttons, loading indicators (shared with layer dependencies)
- **React-native-vector-icons:** Camera controls, scan indicators (shared with layer dependencies)
- **Custom overlay components:** Scan frame, alignment guides

The Camera Module will be implemented with feature detection to ensure graceful degradation on devices without adequate camera capabilities, offering manual entry as a fallback option.

3.10.4 SUBSYSTEM PROGRAMMING LANGUAGES

The Camera Module subsystem will primarily utilize cross-platform JavaScript and TypeScript for React Native development, with native code integrations for performance-critical tasks:

- **TypeScript (4.x):** Used throughout the subsystem for type safety, barcode data modeling, camera screen props, and API interfaces.
- **JavaScript (ES2021):** General React Native logic, UI rendering, and event handling.

Most native logic is abstracted behind well-maintained React Native libraries, minimizing direct platform-specific development.

3.10.5 SUBSYSTEM DATA STRUCTURES

The Camera Module will define consistent data structures for scanned items and API lookup results.

- **ScanSession:** Manages batch scan sessions, allowing multiple ScannedProduct entries to be captured and processed in sequence.
- **CameraConfig:** Stores user and device preferences, such as flash mode, scan overlay settings, and focus behavior.

- **PermissionStatus:** Tracks the current permission state (authorized, denied, unavailable) for use in conditional rendering and error handling.

Scanned data are temporarily cached in memory or local storage, then synchronized to inventory once confirmed by the user or matched to a known product.

3.10.6 SUBSYSTEM DATA PROCESSING

The Camera Module will involve several real-time and asynchronous data processing operations to ensure fast, reliable barcode capture, and minimal user friction. Key processing strategies include

- **Real-Time Frame Analysis:**
 - Using react-native-vision-camera and the vision-camera-code-scanner plugin, camera frames are analyzed on-device for barcode patterns.
 - Barcode detection occurs at up to 60 fps, using optimized native frame processors to prevent UI lag.
- **External Product Lookup:**
 - Once a valid barcode is detected, the system sends a query to external APIs such as Open Food Facts or the UPC Database.
 - The response is parsed into the 'ScannedProduct.metadata' structure.
 - If the product is not found, the user is prompted to enter the details manually.
- **Normalization of nutrition data:**
 - API responses vary by source, so normalization functions are applied to standardize nutrition fields into consistent units (e.g., grams, kcal).
 - Missing or partial values are flagged for user review.
- **Fallback Handling:**
 - If the camera is unavailable or barcode detection fails, users are routed to a manual entry screen pre-filled with the scanned image (if available).

The Camera Module will be designed to minimize user intervention by prioritizing speed, automation, and low-latency feedback, ensuring efficient multi-item scanning in practical kitchen settings.

4 APPLICATION LOGIC LAYER SUBSYSTEMS

4.1 OVERVIEW OF APPLICATION LOGIC LAYER

The Application Logic Layer (ALL) is responsible for coordinating the flow of information between the UI and the Data Layer. It performs:

- Data validation and transformation.
- Business rule enforcement (e.g., filtering allergens).
- AI-driven recipe generation.
- Notification scheduling.
- Secure communication with external APIs.

This layer is implemented using Node.js (v18) and the Express.js framework, ensuring scalability, modularization, and asynchronous operation. Each subsystem is represented as a self-contained module under the `/controllers` directory of the backend.

4.2 LAYER HARDWARE

There are no dedicated hardware components associated with the Application Logic Layer. All logic modules are purely software-based and execute on general-purpose computing hardware, such as:

- Developer machines (macOS, Windows, or Linux) for testing and local execution.
- Cloud-based virtual servers (e.g., AWS EC2, Render, or DigitalOcean) for deployment.

These environments provide sufficient computational power for handling API requests, running the Node.js runtime, and processing AI-based recipe generation requests. Since MindMeal is a mobile application, the Application Logic Layer functions as a remote backend web service, not an embedded or hardware-dependent system.

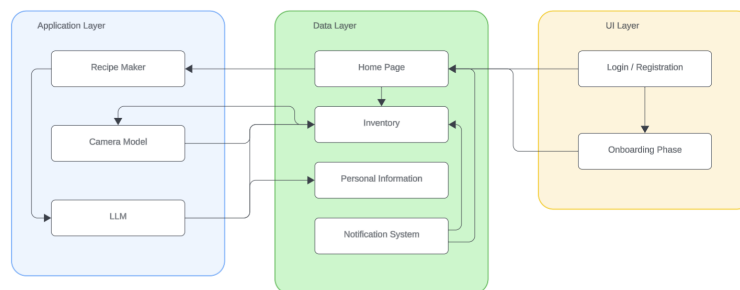


Figure 9: Flow Diagram

4.3 LAYER OPERATING SYSTEM

The Application Logic Layer operates on a Linux-based server OS in production (Ubuntu 20.04 LTS or higher) and is compatible with macOS and Windows environments for development.

Key OS-level features utilized:

- Networking stack: Manages HTTPS communication between client and server.
- File system APIs: Used for logging, temporary storage, and configuration.

- Process management: For asynchronous job execution and background task scheduling.

No specialized or real-time operating systems are required.

4.4 LAYER SOFTWARE DEPENDENCIES

Core frameworks and libraries used throughout this layer:

- Node.js v18+ - primary runtime environment
- Express.js 4+ - HTTP server framework for routing and middleware
- bcrypt - password hashing for user authentication
- jsonwebtoken (JWT) - token-based session management
- Axios / node-fetch - HTTP clients for API communication
- node-cron - scheduling background jobs (notifications)
- sqlite3 / better-sqlite3 - lightweight database access library
- dotenv - environment variable management

4.5 AUTHENTICATION LOGIC

The Authentication Logic subsystem is a web service module implemented in Node.js using Express. It authenticates users by validating login credentials, registering new users, issuing JWT tokens, and managing secure sessions. It acts as the security gateway for all other backend endpoints, ensuring that only verified users can access data or interact with recipes, notifications, and personal profiles. This subsystem communicates with the UI Layer via RESTful API calls and with the Data Layer for credential verification and token persistence.

4.5.1 SUBSYSTEM OPERATING SYSTEM

Runs on the same OS as the parent layer (Linux/macOS). The OS provides networking and cryptographic APIs required for HTTPS communication.

4.5.2 SUBSYSTEM SOFTWARE DEPENDENCIES

- Express.js - API routing
- bcrypt - password hashing
- jsonwebtoken - token generation and verification
- sqlite3 - database driver for user storage

4.5.3 SUBSYSTEM PROGRAMMING LANGUAGES

- JavaScript (Node.js runtime)
- JSON for data serialization

4.5.4 SUBSYSTEM DATA STRUCTURES

User Object `"user_id": "UUID", "email": "string", "password_hash": "string", "created_at": "timestamp"`
JWT Payload `"id": "UUID", "iat": "timestamp", "exp": "timestamp"`

4.5.5 SUBSYSTEM DATA PROCESSING

Algorithm Overview:

- Receive email and password from UI.
- Validate input format.
- Retrieve user record from database.
- Compare password hashes using bcrypt.
- If valid - issue JWT and return to client.
- Otherwise - return error response.

4.6 ONBOARDING LOGIC

The Onboarding Logic subsystem serves as the bridge between new user interaction and persistent profile initialization within the MindMeal ecosystem. Its fundamental role is to collect, validate, and store individual user attributes that directly influence the personalization engine of the application. These attributes include age, allergies, dietary goals, and medical conditions such as ADHD that may affect nutritional needs and cognitive preferences.

From a design perspective, this subsystem is implemented as a modular RESTful API service developed in Node.js and Express.js. It operates as a stateless component, ensuring that every onboarding transaction (registration, update, or deletion) is atomic and isolated. This design principle ensures data consistency even under concurrent access. The collected data is propagated to other critical subsystems, such as the Recipe Engine for meal recommendation filtering and the Notification Logic for reminder customization.

The Onboarding Logic subsystem prioritizes data integrity and accessibility by implementing strict schema validation, encryption, and transactional updates in the database. Its modular structure allows future extension to include additional user data (e.g., fitness levels or mood tracking) without affecting existing APIs.

4.6.1 SUBSYSTEM OPERATING SYSTEM

The subsystem operates on Ubuntu Server 20.04 LTS, leveraging its networking and memory management capabilities to handle concurrent API requests. For local development, macOS Ventura and Windows 11 environments are supported. The operating system provides secure socket layer (SSL) support for encrypted HTTPS communication during onboarding submissions.

4.6.2 SUBSYSTEM SOFTWARE DEPENDENCIES

- **Express.js:** Defines routing and middleware for onboarding endpoints
- **Joi:** Performs schema-based validation for user-submitted profile data
- **sqlite3:** Provides lightweight data persistence for user profiles
- **dotenv:** Secures environment variables such as database credentials
- **bcrypt:** Encrypts sensitive user details before storage (where applicable)

4.6.3 SUBSYSTEM PROGRAMMING LANGUAGES

The subsystem is developed in JavaScript (ES6) using the Node.js runtime environment. All communication between the UI and backend follows JSON (JavaScript Object Notation) format to ensure consistent and lightweight data serialization across the mobile and server layers.

4.6.4 SUBSYSTEM DATA STRUCTURES

The userprofile object represents the structured format of onboarding data. It ensures data uniformity across subsequent modules. "user_id" : "UUID", "age" : 25, "allergies" : ["peanut", "soy"], "dietary_goals" : ["highprotein", "lowsugar"], "medical_conditions" : ["ADHD"]

4.6.5 SUBSYSTEM DATA PROCESSING

Algorithm Description: Upon receiving onboarding data from the UI layer, the subsystem performs a multi-stage process:

- **Validation:** Ensures all fields meet type and boundary constraints.
- **Transformation:** Converts arrays into normalized database entries.
- **Persistence:** Inserts or updates user records atomically.
- **Propagation:** Signals completion to dependent modules (Recipe Engine, Notification Logic).

This flow guarantees robustness against partial writes, ensuring user information remains synchronized across all application layers.

4.7 RECIPE ENGINE

The Recipe Engine is the computational core of MindMeal. It is designed as an intelligent, service-oriented module that synthesizes user preferences, inventory data, and contextual input to generate personalized recipe suggestions. This subsystem employs OpenAI's API for natural language understanding and recipe formulation, augmented with in-house filtering logic to ensure recommendations align with user health and dietary needs.

4.7.1 SUBSYSTEM HARDWARE

The subsystem operates on standard cloud-based compute resources with access to external APIs. All AI-related computations occur on OpenAI's infrastructure, while the MindMeal backend handles orchestration, validation, and response formatting. No local GPU or specialized accelerator hardware is required.

4.7.2 SUBSYSTEM OPERATING SYSTEM

The engine runs on Ubuntu 20.04 LTS, which provides a reliable and secure platform for managing concurrent HTTP requests. API communication with external services utilizes the OS's built-in networking stack, ensuring low-latency data exchange through HTTPS connections.

4.7.3 SUBSYSTEM SOFTWARE DEPENDENCIES

- **Axios:** Facilitates HTTP requests to OpenAI API
- **OpenAI SDK:** Provides AI model integration
- **sqlite3:** Stores recipe caches and previous results
- **dotenv:** Manages API keys securely
- **crypto:** Hashes prompt data for cache lookups

4.7.4 SUBSYSTEM PROGRAMMING LANGUAGES

The Recipe Engine is written in JavaScript (Node.js) with asynchronous programming constructs (async/await) to handle non-blocking API calls. The AI prompt and response utilize JSON as the data interchange format.

4.7.5 SUBSYSTEM DATA STRUCTURES

Recipe data returned from OpenAI follows a normalized schema for integration with the UI Layer: "title": "Spinach Omelette", "ingredients": ["Eggs", "Spinach", "Cheese"], "instructions": "Whisk eggs and cook with cheese.", "nutrition": {"calories": 230, "protein": "14g"}

4.7.6 SUBSYSTEM DATA PROCESSING

Algorithm Description: The subsystem's processing can be divided into five stages:

- **Context Preparation:** Retrieves user profile and inventory data.
- **Prompt Construction:** Concatenates relevant data into an AI-friendly query format.
- **Model Interaction:** Submits the prompt to OpenAI API and receives response.
- **Parsing:** Validates and formats the JSON output into internal structure.
- **Persistence:** Saves results in SQLite for quick retrieval in subsequent sessions.

This subsystem embodies separation of concerns, where AI reasoning remains external, and validation, filtering, and storage remain internal for security and control.

4.8 NOTIFICATION LOGIC

The Notification Logic ensures users remain engaged and informed by managing the creation, scheduling, and delivery of alerts. These alerts include reminders about inventory expiration, personalized meal suggestions, and motivational cues (e.g., hydration reminders). Designed as a background service that runs independently of user sessions, this subsystem guarantees reliability even when the mobile app is inactive.

Notifications enhance the ADHD-friendly design by providing timely, context-aware prompts that reduce the cognitive burden of remembering routine tasks. This subsystem interacts directly with the Inventory Management and Recipe Engine modules to determine when and why notifications should be triggered.

4.8.1 SUBSYSTEM OPERATING SYSTEM

Hosted on Ubuntu 20.04 LTS or equivalent, leveraging OS-level cron-like scheduling to run periodic background checks. Push delivery relies on secure HTTPS endpoints provided by the Expo Notification service.

4.8.2 SUBSYSTEM SOFTWARE DEPENDENCIES

- **node-cron:** Scheduling automated jobs
- **Expo Push SDK:** Sending mobile notifications
- **sqlite3:** Storing notification history
- **dotenv:** Managing configuration
- **moment.js:** Handling date/time calculations

4.8.3 SUBSYSTEM DATA STRUCTURES

A Notification Queue structure temporarily holds pending alerts before dispatch:

```
const notificationQueue = [ {userId: 1001, message: "Milk expires in 2 days", type: "inventory" },
                             {userId: 1002, message: "Try this new smoothie!", type: "recipe" } ];
```

4.8.4 SUBSYSTEM DATA PROCESSING

Algorithm Workflow:

- **Trigger:** Cron scheduler runs every 12 hours.
- **Query:** Retrieves expiring inventory and new recipe suggestions.
- **Queue:** Creates structured notification objects.
- **Dispatch:** Sends messages through Expo Push API.
- **Record:** Marks notification as sent in the database.

This design ensures resilience, enabling queued retry attempts for failed deliveries and future support for AI-personalized timing (e.g., based on user engagement patterns).

4.8.5 INVENTORY MANAGEMENT LOGIC

The Inventory Management Logic governs all operations related to ingredient tracking within the Mind-Meal application. It is responsible for storing, updating, and deleting food items, maintaining stock quantities, and flagging items nearing expiration. By providing an accurate reflection of user pantry data, this subsystem forms a foundational element in the recipe recommendation process and the notification scheduler.

4.8.6 SUBSYSTEM OPERATING SYSTEM

Runs on Ubuntu 20.04 or equivalent. File I/O and time-based triggers rely on system-level features for efficient scheduling.

4.8.7 SUBSYSTEM SOFTWARE DEPENDENCIES

- **Express.js:** RESTful API routing
- **sqlite3:** Persistent data storage
- **Joi:** Data validation
- **node-cron:** Expiration checking
- **dotenv:** Configuration management

4.8.8 SUBSYSTEM PROGRAMMING LANGUAGES

JavaScript (Node.js) for backend logic and JSON as data exchange format.

4.8.9 SUBSYSTEM DATA STRUCTURES

```
"item_id" : "UUID", "user_id" : "UUID", "name" : "Milk", "quantity" : 2, "expiry_date" : "2025 - 10 - 20", "category" : "Dairy"
```

4.8.10 SUBSYSTEM DATA PROCESSING

Algorithm Description:

- Validate user input and ensure proper formatting.
- Insert or update inventory record in SQLite database.
- Invoke checkExpiry() to compare the current date with the expiration threshold.
- Notify Notification Logic subsystem if expiration is within 2 days.

4.9 PERSONAL INFORMATION MANAGER

The Personal Information Manager (PIM) acts as the centralized repository for user-specific data required across various MindMeal modules. It ensures that sensitive user information, such as allergies, medical conditions, and goals, is securely stored and efficiently retrieved for use by other subsystems. This module is pivotal in maintaining data privacy and integrity while providing fast access to relevant data structures.

4.9.1 SUBSYSTEM OPERATING SYSTEM

It depends on Linux/macOS OS-level encryption APIs for secure cryptographic operations.

4.9.2 SUBSYSTEM SOFTWARE DEPENDENCIES

- **sqlite3:** Persistent profile storage
- **crypto:** AES-256 encryption/decryption
- **Express.js:** REST API integration
- **dotenv:** Secure environment variable handling

4.9.3 SUBSYSTEM PROGRAMMING LANGUAGES

JavaScript (ES6), leveraging Node.js asynchronous capabilities.

4.9.4 SUBSYSTEM DATA STRUCTURES

```
"user_id": "UUID", "allergies": ["gluten"], "conditions": ["ADHD"], "dietary_goals": ["lowsugar", "highprotein"]
```

4.9.5 SUBSYSTEM DATA PROCESSING

Algorithm Description:

- Retrieve encrypted user data from database.
- Decrypt payload using AES-256 with a secure environment key.
- Parse decrypted JSON for use in dependent modules.
- Upon updates, re-encrypt payload and overwrite old record.

The design ensures data confidentiality, integrity, and compliance with privacy standards. The encryption layer prevents unauthorized data access, maintaining user trust and system credibility.

5 DATA LAYER SUBSYSTEMS

The Data Layer of the MindMeal system is responsible for managing all forms of persistent storage, synchronization, and secure data transmission between the mobile application and backend services. It ensures the user's personal, dietary, and inventory data are safely stored and remain accessible across sessions, even when offline. This layer abstracts database operations and provides clean, asynchronous data access methods to the Application Logic Layer. It integrates local storage (for offline-first use) and Firebase cloud services (for authentication, inventory data, images, and notifications), ensuring high reliability and data consistency.

5.1 LAYER HARDWARE

The Data Layer primarily operates on mobile devices and cloud infrastructure.

Client Device Hardware:

The app runs on Android and iOS mobile devices. Each device stores user data locally (e.g., inventory and preferences) using AsyncStorage or SQLite for quick access. This allows the user to interact with the app even when offline.

Cloud Infrastructure (Backend):

The MindMeal backend uses Firebase Cloud Firestore as the central NoSQL database for persistent data, Firebase Cloud Storage for image uploads (e.g., food photos and profile pictures), and Firebase Cloud Messaging (FCM) for real-time updates and reminders. The backend is hosted on Google's cloud infrastructure, ensuring scalability, data security, and low-latency access for mobile users.

5.2 LAYER OPERATING SYSTEM

The Data Layer operates across two platforms:

- **Mobile Client:**
 - Android (API 26 and above) and iOS (version 14 and above).
 - Uses platform-specific storage APIs for file handling and caching.
 - Supports background synchronization when the device regains network connectivity.
- **Backend Server:**
 - Runs on Firebase Cloud Platform, which is managed under a Linux-based environment.
 - Python FastAPI (or Node.js Express) services handle backend logic and API requests.
 - No dedicated OS installation is required from the development team, as Firebase abstracts most system-level operations.

5.3 LAYER SOFTWARE DEPENDENCIES

Component	Function
Firestore	Cloud database for storing user, inventory, and chat data.
Authentication	Secure login and registration services integrated with frontend.
Cloud Storage	Manages image uploads and file handling for food photos.
Cloud Messaging (FCM)	Handles push notifications and reminder delivery.
AsyncStorage / SQLite	Local data storage for offline mode and cache management.
Axios / Fetch API	Facilitates secure REST communication between app and back-end.
Python FastAPI / Node.js Express	Handles REST endpoints, AI communication, and business logic.

Table 2: Key Components and Their Functions in the MindMeal System

5.4 DATA MANAGEMENT SUBSYSTEM

The Data Management Subsystem provides structured, modular access to user data and supports synchronization between local storage and Firebase Cloud Firestore. It manages CRUD (Create, Read, Update, Delete) operations for user profiles, inventory items, chat sessions, and reminders, maintaining seamless data flow between the frontend and backend.

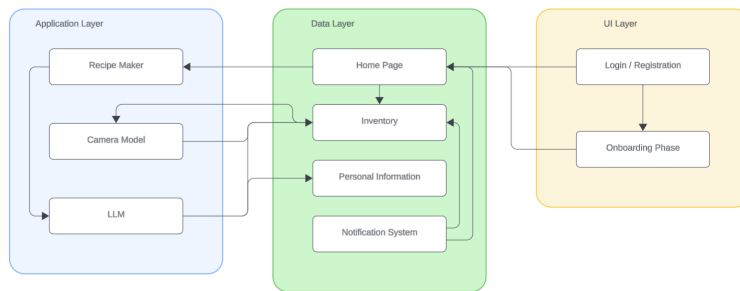


Figure 10: Flow Diagram

5.4.1 SUBSYSTEM HARDWARE

Operates directly on the user's mobile device for local data handling.
Connects to Firebase cloud storage and Firestore for remote data management.
No additional on-premise or external hardware is required.

5.4.2 SUBSYSTEM OPERATING SYSTEM

Client: Android/iOS, using React Native's native APIs for secure data handling.

Server: Cloud-hosted Firebase infrastructure with Python or Node.js backend for business logic and API processing.

5.4.3 SUBSYSTEM SOFTWARE DEPENDENCIES

Firebase SDKs (Firestore, Auth, Storage, Messaging) for all database and communication tasks.

React Native AsyncStorage and SQLite for offline-first data management.

Axios for HTTP requests to backend and external APIs (e.g., recipe LLM).

Expo FileSystem for file access and caching.

Dotenv for secure environment variable management.

5.4.4 SUBSYSTEM PROGRAMMING LANGUAGES

Frontend: JavaScript (React Native).

Backend: Python (FastAPI) or Node.js (Express).

Data Exchange Format: JSON for structured communication between client and server.

Database Schema: NoSQL (Firestore collections) and SQLite tables for local cache.

5.4.5 SUBSYSTEM DATA STRUCTURES

The primary data structures managed within the Data Layer are designed to store and synchronize user information, food inventory details, and chat interactions. Each structure ensures efficient data access and consistency between local storage and Firebase Cloud Firestore.

- **User Profile Data Structure:**
Stores individual user information such as name, email, dietary goals, allergies, notification preferences, and reminder settings. It is used for personalization and authentication.
- **Inventory Item Data Structure:**
Represents food items added by the user, including item name, quantity, category, expiration date, and image reference. Used for managing the food inventory and suggesting meal ideas.
- **Recipe Chat History Data Structure:**
Contains records of user prompts and system-generated recipe or meal suggestions. Used to maintain chat context and provide personalized recommendations.
- **Onboarding Data Structure:**
Temporarily stores user input during the registration and setup process, including dietary preferences, restrictions, and initial goals. Used to customize the app experience upon first login.
- **Notification Settings Data Structure:**
Stores user-specific reminder configurations such as time, frequency, and enabled channels. Used by the Firebase Cloud Messaging service to schedule and deliver notifications.
- **Analytics and Activity Log Structure:**
Captures user actions such as added inventory items, opened recipes, and completed onboarding steps. Used for tracking engagement and improving future updates.

5.4.6 SUBSYSTEM DATA PROCESSING

The Data Processing subsystem defines how information flows between the local storage on the mobile device and Firebase Cloud Firestore. Its purpose is to ensure smooth synchronization, efficient data retrieval, and consistent updates across sessions and devices.

- **Data Input and Capture:**
User actions such as adding inventory items, updating profiles, or sending chat prompts trigger data collection through input fields or interface interactions.
- **Local Storage Operations:**
Data is first stored in local storage (SQLite or AsyncStorage) to guarantee offline availability and quick access without depending on network connectivity.
- **Cloud Synchronization:**
When a stable internet connection is detected, locally stored changes are synchronized automatically with Firebase Firestore and Cloud Storage through secure REST APIs.
- **Conflict Resolution:**
Timestamp-based version control ensures that the most recent update takes precedence during synchronization, preventing data inconsistency between the client and server.
- **Data Retrieval and Caching:**
Frequently accessed data such as inventory items, chat history, and profile details are cached locally to reduce load time and minimize API calls.
- **Error Handling and Logging:**
Any failed synchronization or API errors are logged locally, retried using an exponential backoff strategy, and reported for debugging and maintenance purposes.

This subsystem ensures reliability, speed, and data consistency across different devices while maintaining a seamless user experience even during periods of limited connectivity.

6 APPENDIX A

Include any additional documents (CAD design, circuit schematics, etc) as an appendix as necessary.

REFERENCES